

INGENIERIA DE SOFTWARE I
(TA046) CURSO DIEGO MONTALDO

Trabajo Práctico

Sistema de gestión para comedores universitarios

× ×

-

Rodrigo Rotondo	109210
Tomas Conti	111760
Daniela Gomez	107652
Valentin Marturet	104526
Santiago Henseler	110732
Agustin Pelliciari	108172

1. Alcance

1.1. Propósito del sistema

Nuestro sistema de gestión para comedores universitarios busca modernizar la experiencia de estudiantes y personal, permitiendo a los estudiantes consultar el menú, armar, realizar y pagar pedidos desde una interfaz web, y brindando al personal herramientas para gestionar pedidos, preparación en cocina y disponibilidad de productos en tiempo real. Adicionalmente, facilita la administración del menú, stock, promociones y accesos del personal.

1.2. Alcance funcional

Actores principales

- **Estudiante/cliente:** se registra y autentica, gestiona su perfil, consulta menú y disponibilidad, arma, realiza y paga pedidos, consulta estado y los retira.
- **Personal de cocina/caja:** visualiza pedidos entrantes, cambia estados (p.ej., pedido confirmado, en preparación, completado), gestiona disponibilidad operativa.
- **Encargado/a del comedor:** administra menú, ingredientes, combos/paquetes, stock, promociones, y acceso del personal (roles).

Funciones incluidas

- **Gestión de usuarios:** registro con verificación de email; autenticación y recuperación de contraseña; perfil con nombre, apellido, email, foto, edad y domicilio.
- **Catálogo y composición de productos:**
 - Ítems individuales y *combos* (paquetes con precio especial).
 - Ítems compuestos por ingredientes; dependencia de disponibilidad por ingrediente.
 - Disponibilidad automática: si falta un ingrediente clave, los ítems/combos afectados quedan *no disponibles* para los estudiantes.
- **Gestión de stock:** alta/baja/ajuste de ingredientes y productos; soporte para procesos de reposición (manuales).
- **Promociones dinámicas:** reglas configurables (p.ej., “con un sándwich, un café gratis”, 3x2, descuentos por día o por monto, descuentos condicionados por método de pago).
- **Ciclo de vida del pedido:**
 1. *Pedido armado/pendiente de pago* por el estudiante.
 2. *Confirmado/pagado* por el estudiante.
 3. *En preparación* por personal de cocina.
 4. *Completado / listo para retirar*.

Cancelación permitida mientras no haya comenzado la preparación (estado en preparación o posterior).
- **Backoffice del encargado:** ABM de menú, ingredientes, combos, promociones y gestión de accesos del personal (asignación de roles como administrador/empleado).
- **Pagos:** selección de método de pago (efectivo, débito, crédito).
- **Notificaciones de estado:** avisos al usuario ante cambios relevantes en el estado de sus pedidos (p.ej., cuando pasa a estar en preparación o completado).

1.3. Alcance no funcional

- **Seguridad y autenticación:** autenticación con tokens (JWT), almacenamiento seguro de credenciales, verificación de email y recuperación de contraseña; control de acceso basado en roles (cliente, empleado, administrador).
- **Disponibilidad:** servicio operativo en horario del comedor.
- **Rendimiento:** respuestas rápidas para consultas de menú bajo carga típica; actualización frecuente del estado de los pedidos en las vistas de cocina/pedidos.
- **Consistencia:** para operaciones críticas (estado del pedido, disponibilidad de ingredientes y aplicación de descuentos).
- **Compatibilidad:** API documentada (Swagger/Postman); contenedores Docker para despliegue.

1.4. Stakeholders y usuarios

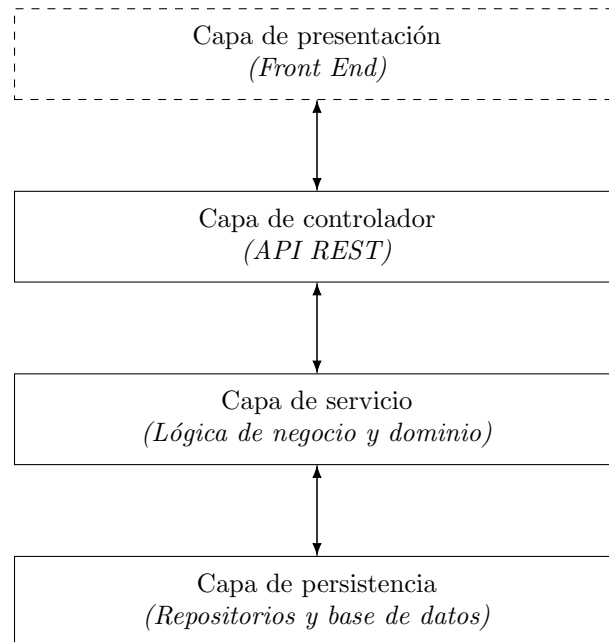
- **Estudiantes** (usuarios finales).
- **Personal del comedor** (cocina/caja).
- **Encargado/a del comedor** (administración y configuración).
- **Equipo de TI/desarrollo** (operación y evolución).

1.5. MVP / Early Testable Product

- Registro, autenticación, validación de email y recuperación de contraseña.
- Menú con disponibilidad por stock e ingredientes.
- Flujo de pedido de extremo a extremo (incluyendo estados de pago, preparación y completado).
- Panel de cocina y backoffice mínimos.
- API y despliegue en Docker para pruebas.

2. Arquitectura de Software

Se decidió utilizar un patrón de arquitectura enterprise, vinculado con el patrón por capas. Se definieron las siguientes capas:



- **Capa de presentación:** encargada de presentar la información a los distintos usuarios y permitir, a partir de eventos generados por el usuario, interactuar con la capa de controlador. Siguiendo la arquitectura enterprise, esta capa brinda vistas distintas dependiendo del tipo de usuario que accede a la aplicación (estudiante/cliente, personal de cocina/caja y encargado/administrador).
- **Capa de controlador:** expone una API REST que recibe las solicitudes de la capa de presentación, realiza el mapeo entre las llamadas HTTP y los casos de uso del sistema, y delega la lógica de negocio en la capa de servicio y negocio. También es responsable de aplicar las políticas de seguridad de la API (autenticación y autorización por rol).
- **Capa de servicio y negocio:** implementa las funcionalidades necesarias para los distintos casos de uso, orquestando las reglas de negocio del dominio (pedidos, menú, descuentos, usuarios, etc.). Esta capa encapsula la lógica de aplicación y utiliza la capa de persistencia para acceder a los datos.
- **Capa de persistencia:** ofrece a la capa de servicio y negocio el servicio de persistir, almacenar y recuperar la información, a través de repositorios que abstraen el acceso a la base de datos relacional.

3. Vista Lógica

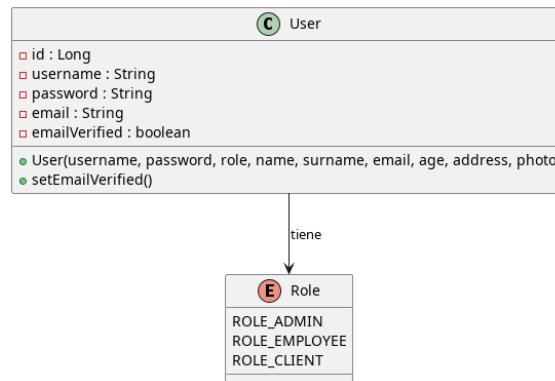


Figura 1: Diagrama de Clases: Usuarios

La entidad **User** representa a cualquier persona que utiliza el sistema (estudiante/cliente, personal del comedor o encargado). Contiene los datos de identidad y autenticación (email, contraseña) y se asocia a un único **Role**, que indica el tipo de usuario y, por lo tanto, los permisos que posee. El enum **Role** distingue entre usuarios administradores (**ROLE_ADMIN**), empleados del comedor (**ROLE_EMPLOYEE**) y clientes finales (**ROLE_CLIENT**). A partir de este rol, la capa de seguridad aplica las políticas de autorización para acceder a los distintos casos de uso del sistema.

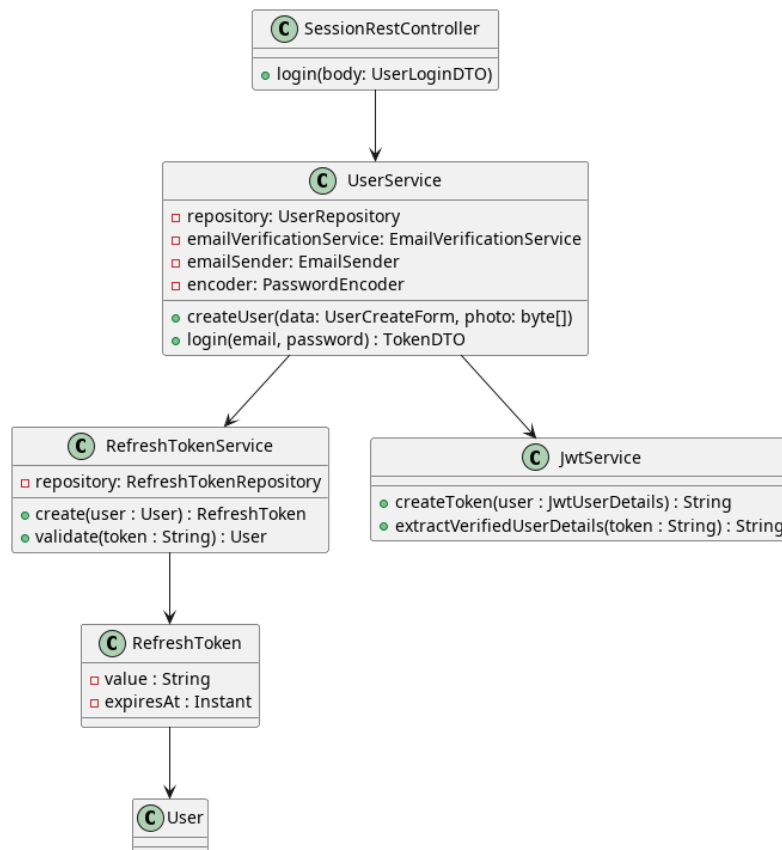


Figura 2: Diagrama de Clases: Autenticacion de usuarios

El subsistema de autenticación se organiza alrededor de `UserService`, que concentra la lógica de inicio de sesión y refresco de tokens. El `SessionRestController` expone los endpoints HTTP de `login` y `refresh` y delega el trabajo en dicho servicio. A partir de las credenciales recibidas, `UserService` valida al `User`, genera un token de acceso JWT mediante `JwtService` y un token de refresco persistido como `RefreshToken` a través de `RefreshTokenService`, devolviendo ambos en un `TokenDTO`. De esta forma, el cliente puede autenticarse usando JWT y renovar su sesión sin reenviar la contraseña.

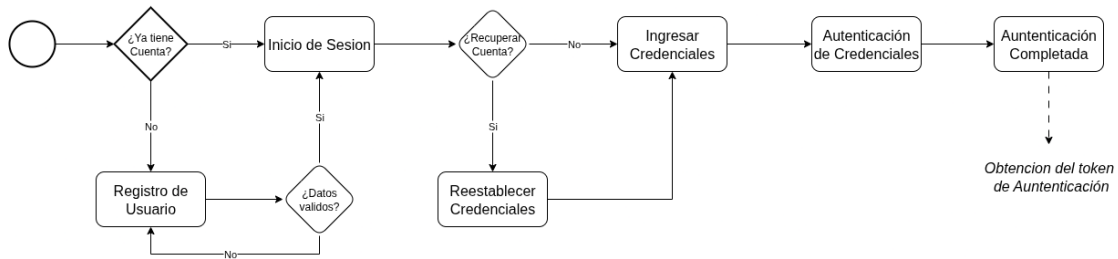


Figura 3: Diagrama de Estados: Autenticacion de un Cliente

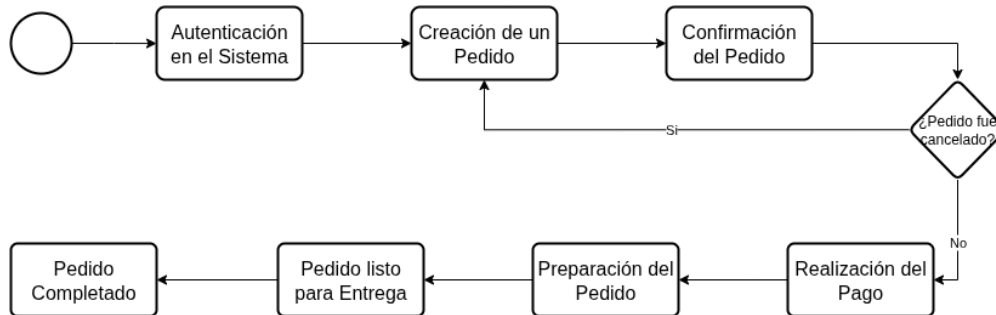


Figura 4: Diagrama de Estados: Flujo de un Cliente

El anterior diagrama ilustra el flujo típico de un cliente dentro del sistema. Primero, el usuario realiza la **autenticación en el sistema** y luego pasa a la **creación de un pedido**, donde selecciona productos y combos. Una vez definido, se realiza la **confirmación del pedido**. En este punto se verifica si el pedido fue cancelado: en caso afirmativo, el flujo retorna a la etapa de creación; en caso negativo, el cliente continúa con la **realización del pago**.

Tras el pago, el pedido pasa a la etapa de **preparación**, luego a **pedido listo para entrega** y finalmente a **pedido completado** una vez que el cliente retira su pedido.

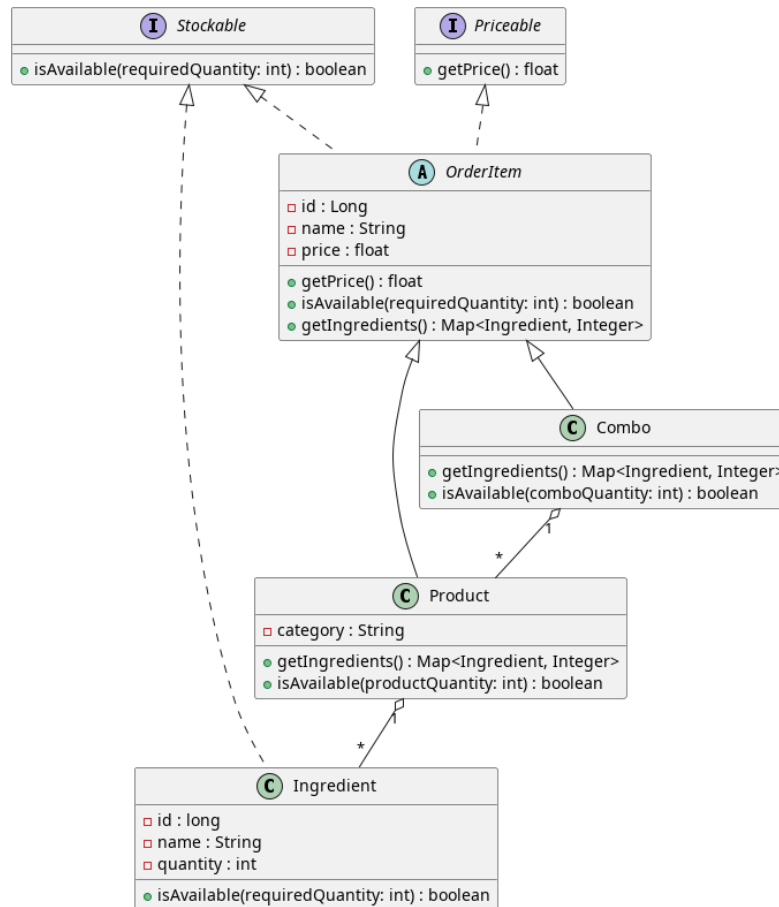


Figura 5: Diagrama de Clases: Stock

El anterior diagrama modela la lógica de stock y precios del sistema. La interfaz **Stockable** abstrae a cualquier elemento cuya disponibilidad pueda consultarse, mientras que **Priceable** representa a los elementos que tienen un precio asociado. **OrderItem** es una clase abstracta que unifica el comportamiento de los ítems que pueden formar parte de un pedido (productos y combos), exponiendo operaciones para conocer su precio, verificar disponibilidad y obtener los ingredientes que consumen. Los **Product** se componen de uno o más **Ingredient**, que son quienes mantienen el stock físico (**quantity**), y los **Combo** se componen de varios **Product**. De esta forma, la disponibilidad de productos y combos se calcula siempre en función del stock real de los ingredientes.

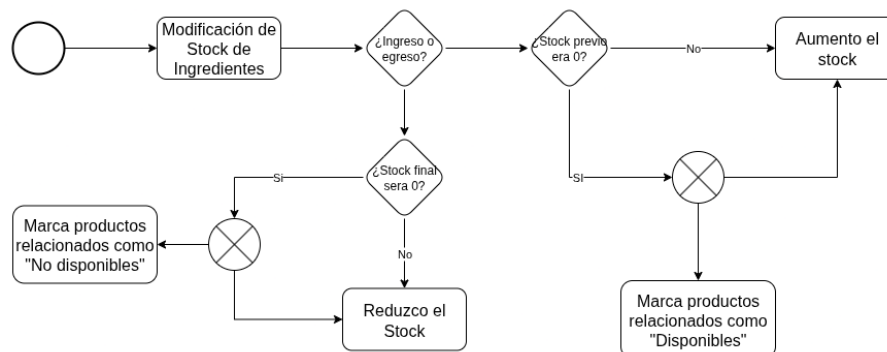


Figura 6: Diagrama de Estados: Stock

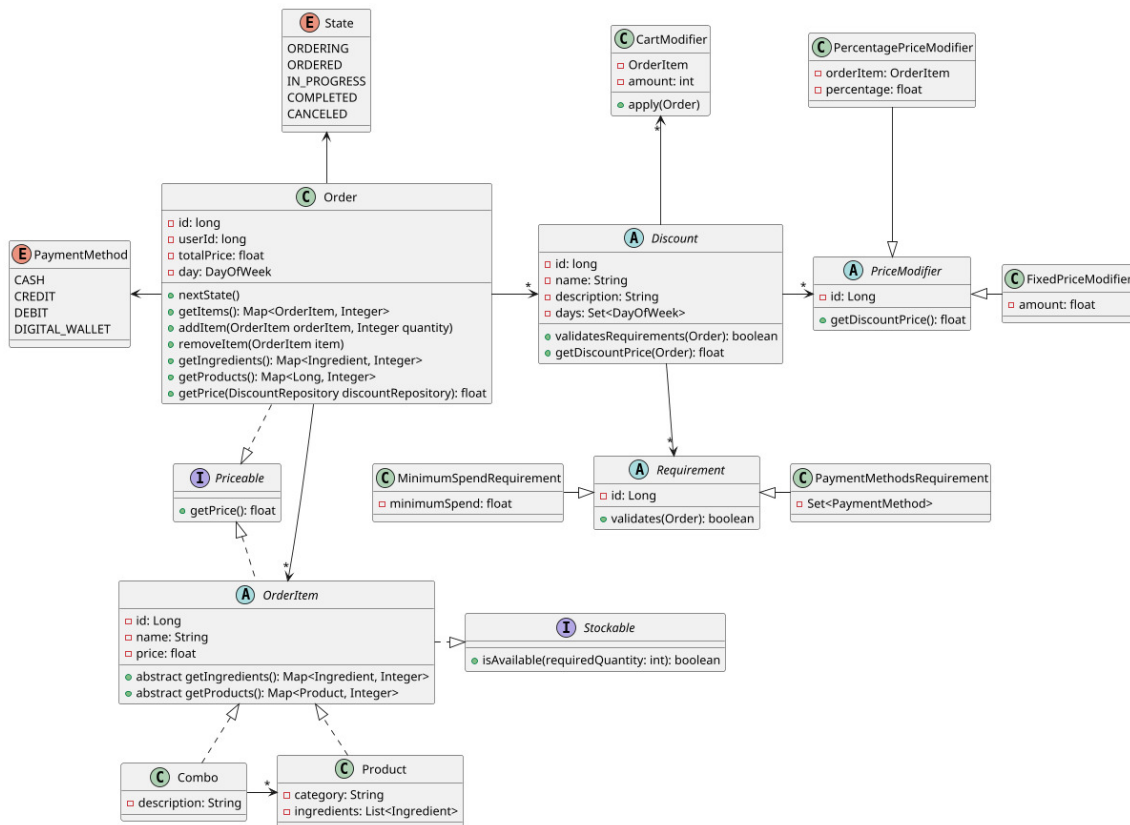


Figura 7: Diagrama de Clases: Estados de una Orden

Este diagrama modela la lógica central de los pedidos y la aplicación de descuentos. La entidad **Order** representa un pedido realizado por un usuario (**userId**), para un determinado día (**day**) y con un estado de avance (**State**: **ORDERING**, **ORDERED**, **IN_PROGRESS**, **COMPLETED**, **CANCELED**). Cada pedido agrupa un conjunto de ítems (**OrderItem**), que pueden ser **Product** o **Combo**, y mantiene el método de pago seleccionado (**PaymentMethod**).

Los productos y combos implementan la interfaz **Priceable** e indirectamente **Stockable**, permitiendo calcular el precio y verificar disponibilidad a partir de sus ingredientes. A nivel de descuentos, la clase abstracta **Discount** define la estructura general: un conjunto de **Requirement** (por ejemplo, **MinimumSpendRequirement** o **PaymentMethodsRequirement**) que determinan cuándo el descuento es aplicable, y uno o más **PriceModifier** (**FixedPriceModifier**, **PercentagePriceModifier**) y **CartModifier** para modificar el precio final o el contenido del pedido. De esta forma, **Order** puede calcular su precio total combinando los ítems y los descuentos que cumplan con los requisitos definidos.

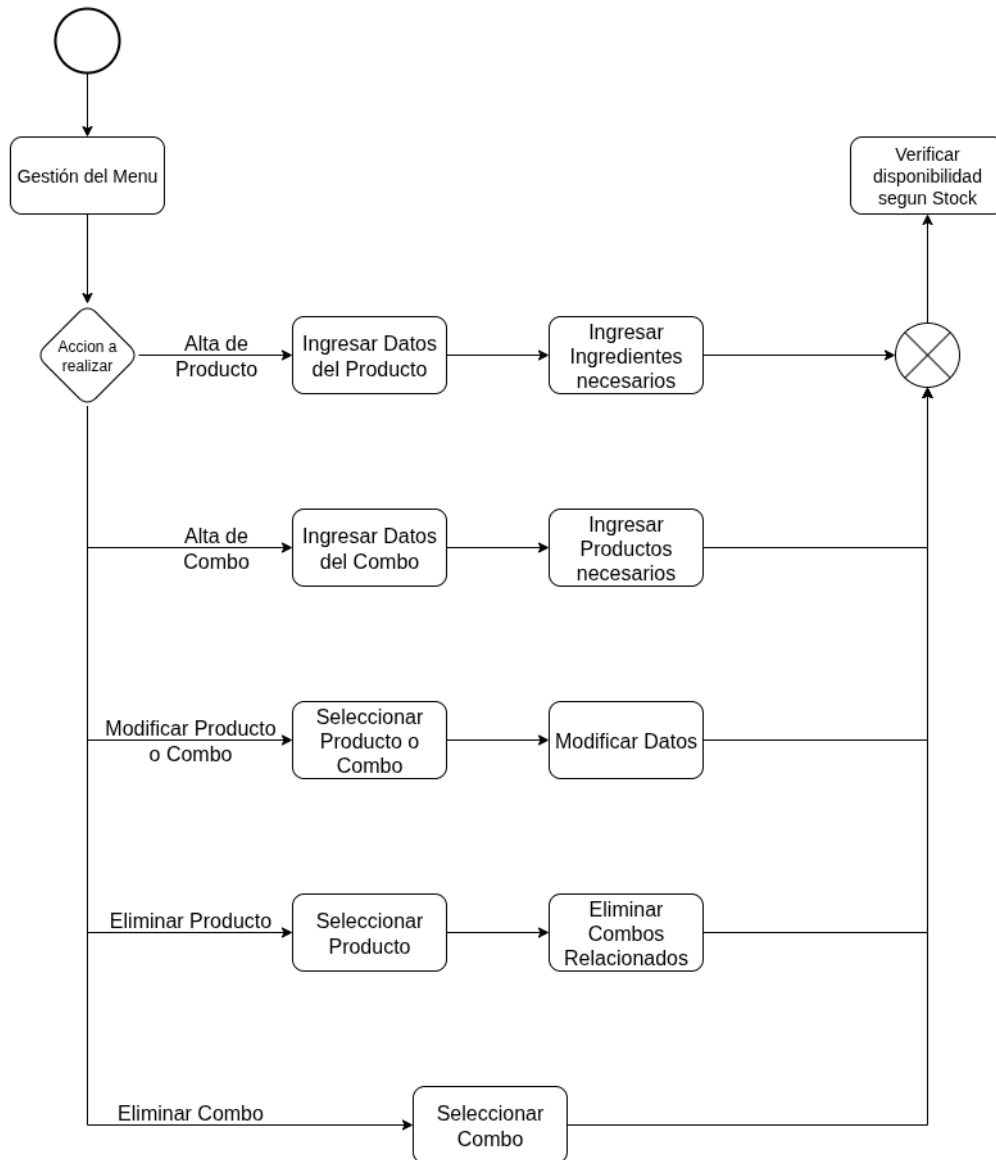


Figura 8: Diagrama de Estados: Productos

Por ultimo, detallamos el flujo para la **gestión del menú** por parte del encargado. A partir de la decisión inicial (“*Acción a realizar*”), se abren distintos caminos:

- **Alta de producto:** se ingresan los datos del producto y luego los ingredientes necesarios.
- **Alta de combo:** se cargan los datos del combo y los productos que lo componen.
- **Modificación de producto o combo:** se selecciona el ítem a modificar y se actualizan sus datos.
- **Eliminación de producto:** se selecciona el producto y, si corresponde, se eliminan los combos relacionados que lo utilizaban.
- **Eliminación de combo:** se selecciona el combo a borrar.

Todos estos caminos confluyen finalmente en una actividad común de **verificar disponibilidad según stock**, donde el sistema recalcula qué productos y combos quedan disponibles en función de los ingredientes y productos vigentes.

4. Vista de Procesos

En la implementación actual se distinguen tres procesos lógicos principales, desplegados como servicios independientes:

- **Cliente:** el navegador del usuario, donde se ejecuta la aplicación web. Desde aquí se realizan las peticiones HTTP al sistema, tanto para cargar la interfaz inicial como para invocar la API (por ejemplo, login, gestión de pedidos, consulta de menú, etc.).
- **Servidor de frontend:** servicio web que sirve los recursos estáticos de la aplicación (HTML, CSS, JavaScript) y actúa como punto de entrada público. Una vez cargada la app web en el navegador, el servidor de frontend sigue atendiendo peticiones de recursos estáticos, pero la mayor parte de la interacción ocurre entre el cliente y la API backend.
- **Servidor backend:** aplicación Spring Boot que implementa las capas de controlador, servicio y persistencia. Expone la API REST que utiliza el cliente para todas las operaciones de negocio (usuarios, autenticación, pedidos, menú, descuentos, etc.) y se comunica con la base de datos relacional.

Adicionalmente existe un proceso de base de datos (PostgreSQL) que persiste el estado del sistema y que es utilizado exclusivamente por el servidor backend. En el diagrama se lo representa como un servicio de soporte para simplificar el foco en la interacción cliente-aplicación.

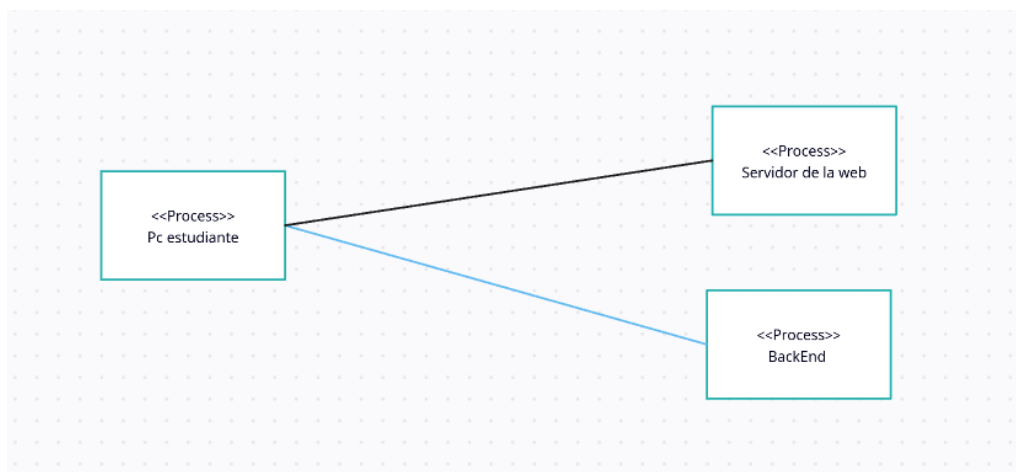


Figura 9: Diagrama de la vista de procesos

5. Vista de Desarrollo

En la vista de desarrollo se distinguen dos proyectos principales, que se corresponden con las carpetas del repositorio:

- **Frontend** (`frontend/`): aplicación web en React que implementa la capa de presentación. Dentro de este proyecto se distinguen la UI (páginas y componentes) y los servicios de acceso a la API (hooks y funciones de `apiFetch`).
- **Backend** (`backend/`): aplicación Spring Boot que implementa las capas de aplicación, dominio e infraestructura. Se organizan los paquetes de controladores REST, servicios de negocio (Aplicación), entidades/agregados (Dominio) y componentes de infraestructura (repositorios, seguridad, envío de emails).

A nivel conceptual, estos proyectos respetan la separación en capas de Presentación, Aplicación, Dominio e Infraestructura descrita en la arquitectura de software.

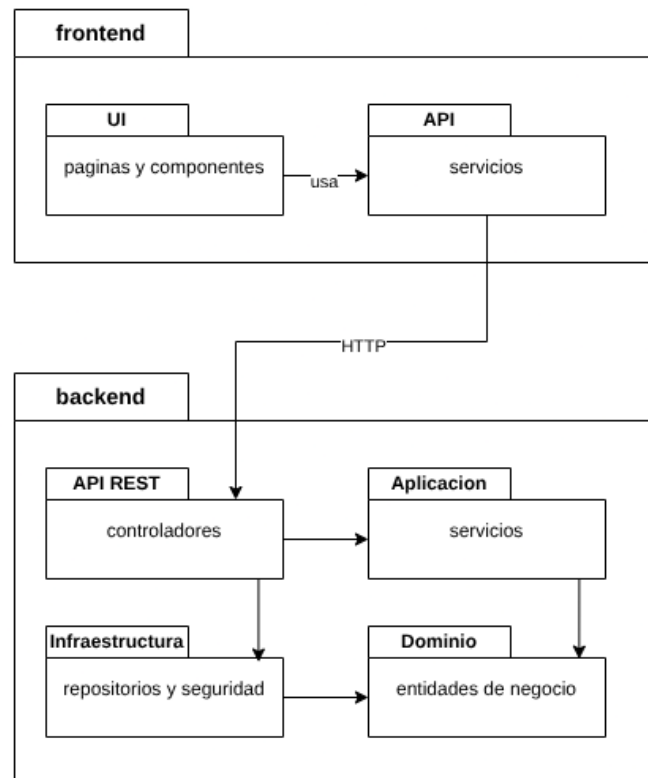


Figura 10: Diagrama de Paquetes

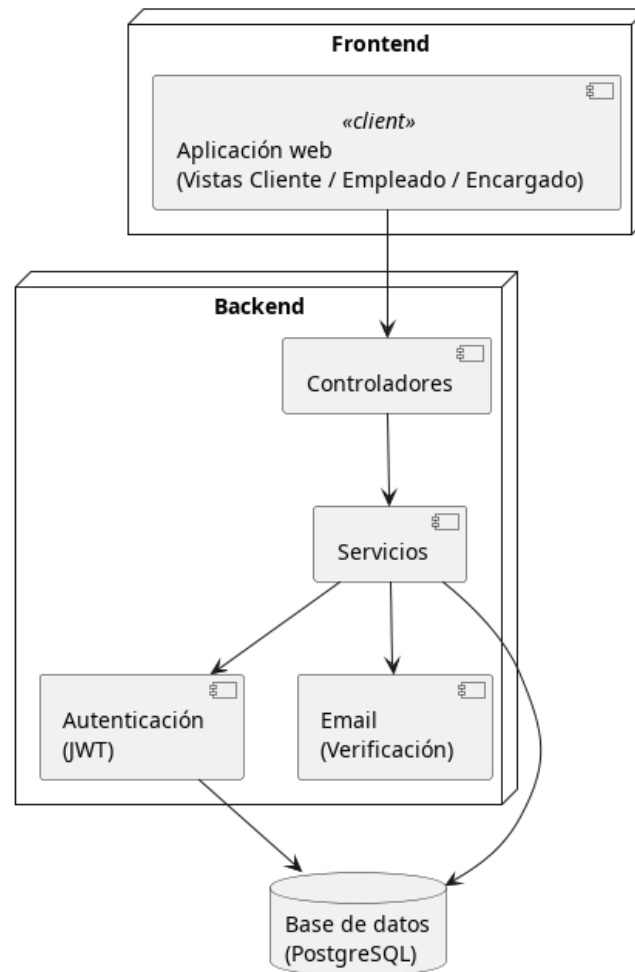


Figura 11: Diagrama de Componentes

El diagrama de componentes muestra la interacción entre la aplicación web frontend y los componentes principales del backend. El **Frontend** está implementado como una SPA en React que ofrece distintas vistas para cliente, empleado y encargado, y se comunica exclusivamente con la **API REST** del backend mediante HTTP.

En el **Backend** se distinguen los controladores de la API, los **servicios de negocio** (pedidos, catálogo, usuarios, descuentos), el módulo de **autenticación** basado en JWT y tokens de refresh, y el módulo de **email** utilizado para verificación de cuenta y recuperación de contraseña. Todos estos componentes acceden a la **base de datos PostgreSQL** a través de repositorios JPA, donde se persisten usuarios, pedidos, productos, ingredientes, menús, descuentos y tokens.

6. Vista Física

Se utiliza una arquitectura web de múltiples capas desplegada sobre contenedores Docker. En el entorno actual, el **frontend** y el **backend** se ejecutan como servicios independientes, junto con un servidor **nginx** (ingress) que actúa como reverse proxy y una instancia de **PostgreSQL** para la base de datos. Todos estos servicios se alojan en un mismo servidor físico, comunicándose entre sí a través de una red interna de Docker.

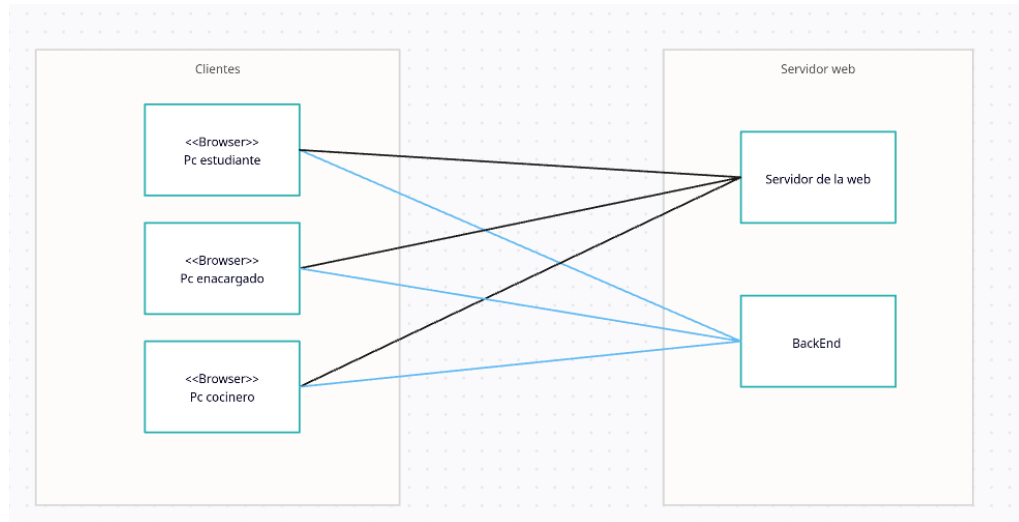


Figura 12: Diagrama de la vista física

El cliente accede a la aplicación mediante un navegador web, realizando peticiones HTTP al servidor **ingress**. En el primer acceso, nginx sirve los recursos estáticos del frontend (HTML, CSS, JavaScript) y el navegador carga la SPA. A partir de ese momento, la aplicación cliente se comunica principalmente con el **servidor backend** a través de la API REST para todas las operaciones de negocio (autenticación, pedidos, menú, descuentos, etc.), mientras que el backend se encarga de acceder a la base de datos en el contenedor de **PostgreSQL**.

7. Escenarios

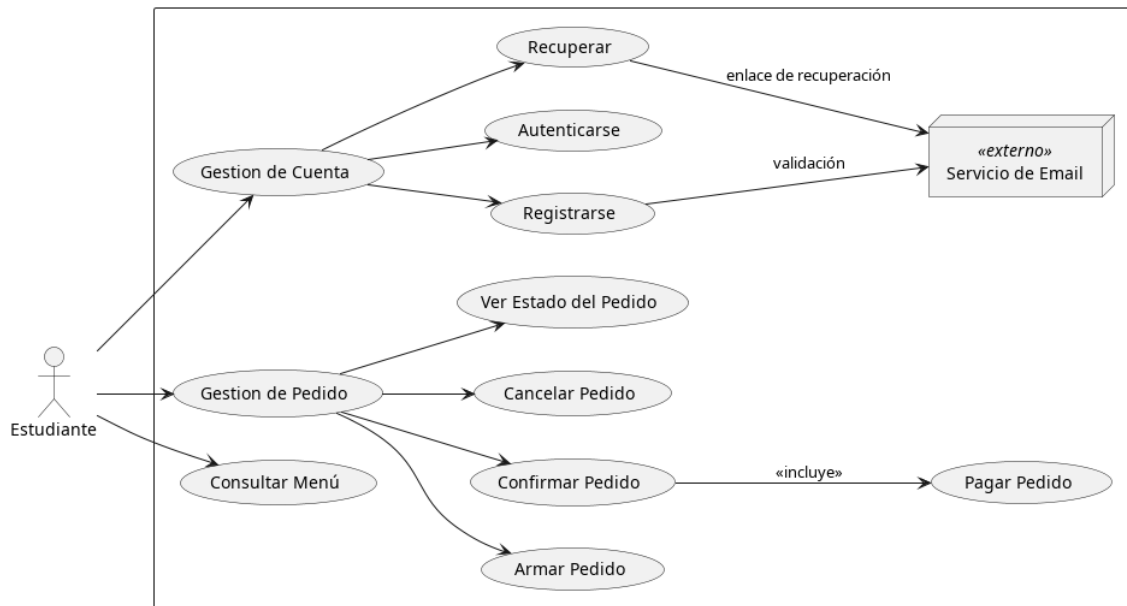


Figura 13: Casos de uso: Estudiantes

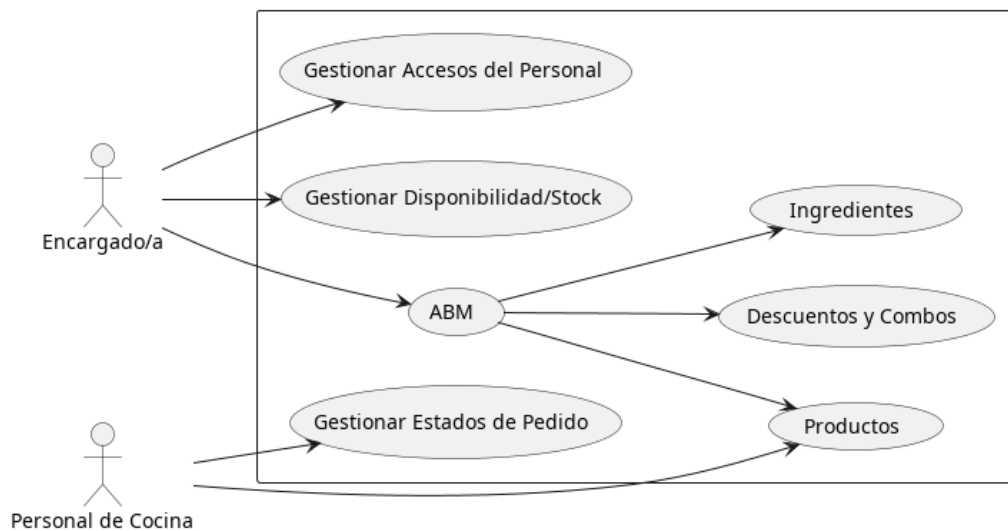


Figura 14: Casos de uso: Personal del comedor